Pricing Options on the IDI Index: Model, Implementation and Verification

FixedIncome Pricing project · Derivatives module
Technical report · 16 October 2026

Abstract. This report describes the design, implementation and verification of a pricer for the B3 option on the IDI index (an interest-rate index that compounds the Brazilian overnight DI rate) within a from-scratch Brazilian fixed-income library. The forward of the index is obtained by cash-and-carry on the bootstrapped DI curve, and the option is priced with the Black (1976) formula on that forward, discounted on the same curve. We derive closed-form Greeks, obtain interest-rate risk from the library's bump-and-reprice engine, recover implied volatility with a safeguarded Newton solver, and supply volatility through a surface that interpolates total variance and rejects calendar arbitrage. The implementation is verified by reference values, put–call parity, finite differences, round-trip implied volatility and an independent Monte Carlo simulation (46 automated tests). Limitations are stated explicitly: the tests use a synthetic index level, no reconciliation with B3 reference premiums has been performed, and butterfly arbitrage is not checked.

Keywords: IDI option; Black-76; implied volatility; volatility surface; calendar arbitrage; DI curve; Brazilian fixed income.

1. Introduction

The **FixedIncome.Derivatives** module prices one product end to end: the B3 **option on the IDI index**. It sits on top of what the library already had (a bootstrapped DI curve, the Bus/252 day count, the `MarketContext` snapshot and the bump-and-reprice risk engines) and adds the first *model* in the repository: a lognormal model for the forward, the Black (1976) formula, a volatility surface, Greeks and an implied-volatility solver.

Everything before this module was *static*: a bond or a swap has known cash flows, so its price is a sum of discounted amounts and no opinion about the future is needed beyond the curve. An option is the first instrument where the payoff depends on the *distribution* of the underlying, so a new market input appears: volatility. This report documents that step. Its contributions are:

- a derivation of the IDI forward by cash-and-carry on the DI curve, so that one curve both projects the forward and discounts the payoff (Section 3);
- pricing with the Black (1976) formula, closed-form Greeks, and interest-rate risk obtained from the existing bump-and-reprice engine (Sections 4–5);
- an implied-volatility solver (safeguarded Newton) and a volatility surface interpolated in total variance with a calendar-arbitrage check (Sections 6 and 8);
- a verification strategy combining reference values, identities, finite differences and an independent Monte Carlo computation (Section 10).

Section 2 describes the contract; Section 7 documents the normal distribution function on which every result depends; Section 9 shows how the pieces connect to the library's market snapshot; Section 11 states the limitations. Derivations are collected in Appendix A.

1.1 Software architecture: model, market, instrument, engine

The module separates four concerns. Keeping them apart is what makes each testable on its own:

Table 1. Layers of the module and what each one knows about.

Layer	Files	Knows about	Does not know about
Model (pure maths)	`Models/NormalDistribution.cs` `Models/Black76.cs`	numbers: F, K, σ, T, discount factor	dates, curves, calendars, B3
Market objects	`Volatility/IVolSurface.cs` `FlatVolSurface.cs`, `GridVolSurface.cs`	σ as a function of (T, log-moneyness)	the instrument, the curve
Instrument (data)	`Options/IdiOption.cs` `Options/OptionType.cs`	strike, maturity, call/put, payoff	how it is priced
Engine (glue)	`Pricing/IdiOptionPricingEngine.cs`	reads the MarketContext, builds F, T, σ, calls the model	the internals of the formula

This mirrors how a quant library is organised on a desk: **models are stateless functions** that can be unit-tested against textbook values; **market objects** are snapshots; the **engine** is the only place that knows how a product maps onto a model. When a number looks wrong, the layering tells you where to look first.

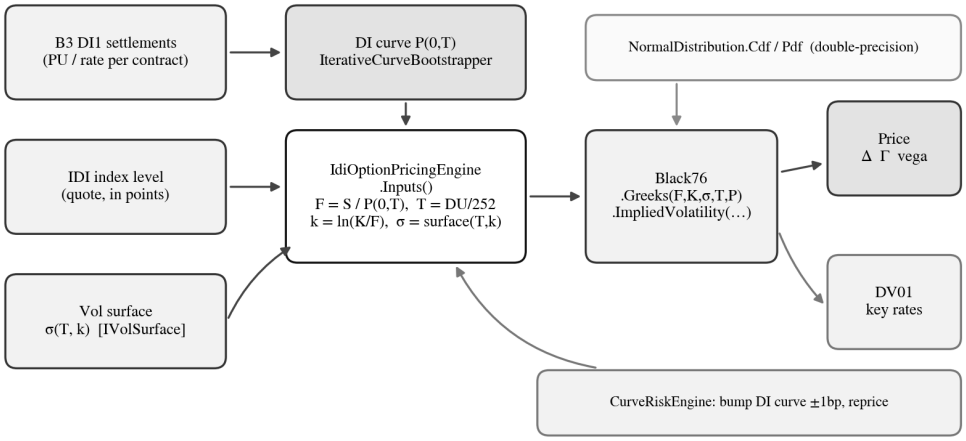


Figure 1. Data flow from market inputs to price and risk. Shaded boxes are market objects and outputs, the white box is the engine that connects them, and the grey boxes at the bottom show how risk is obtained by bumping the curve.

2. The product: an option on the IDI

2.1 The IDI index

The **IDI** (Índice de Taxa Média de Depósitos Interfinanceiros de Um Dia) is a cumulative interest index. B3 starts it at 100,000 points and compounds it every business day with the overnight DI rate:

$$\mathrm{IDI}_t = \mathrm{IDI}_{t-1}(1+d_t), \qquad d_t = (1+\mathrm{DI}_t)^{1/252} - 1 \tag{1}$$

Here $\mathtt{DI}_t$ is the annualised rate (Bus/252) published for day t. As long as rates are positive the IDI only goes up. It is, in effect, **the value of R$ 100,000 rolled overnight in the interbank market since the index was created** – a money-market account expressed in points. That is why its forward has such a clean form (Section 3).

2.2 Contract terms (B3)

Table 2. Terms of the B3 option on the IDI index and their counterparts in the code.

Term	Value	In the code
Style	European, exercised automatically at maturity if in the money	no early-exercise logic; one payoff date
Underlying	IDI index (points)	market quote `QuoteNames.Idi`
Strike and premium	quoted in index points	`IdiOption.Strike`, engine output
Point value	R$ 1.00 per point; contract size 1	`IdiOption.PointValue = 1.0`
Maturity	first business day of the contract month	`MaturityDate` (caller supplies it)

The contract terms follow the B3 product page for options on the IDI (B3, 2026a). The module does not yet parse tickers (compare `DiFuture.FromTicker`), so the maturity date is passed in directly.

2.3 Payoff

$$\text{Payoff}_{\text{call}} = \max(\text{IDI}_T - K,\, 0) \times \text{R\$}\,1, \qquad \text{Payoff}_{\text{put}} = \max(K - \text{IDI}_T,\, 0) \times \text{R\$}\,1 \tag{2}$$

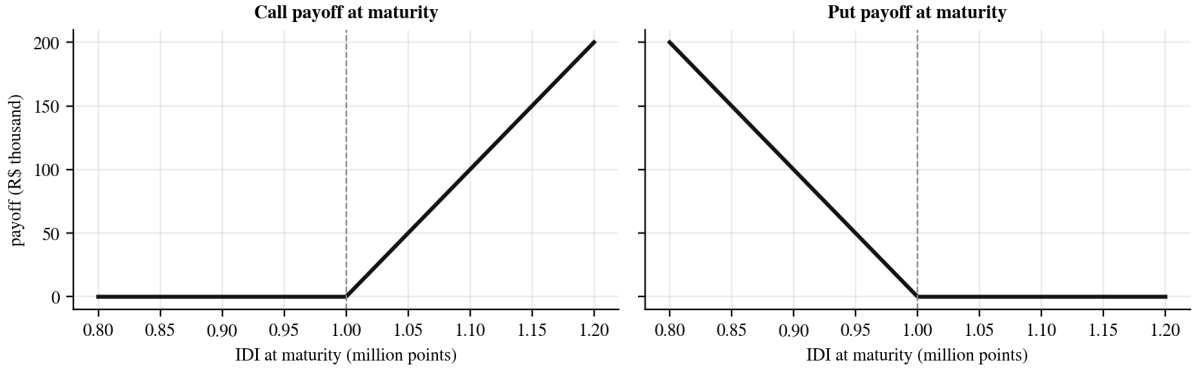


Figure 2. Payoff at maturity for strike K = 1,000,000 points.

Since $\text{IDI}_T = \text{IDI}_0 \times$ (accumulated DI growth until T), a **call** pays when the interest actually accumulated until maturity exceeds the growth that the strike implies, K / IDI_0. In other words the call behaves like a **cap** on the DI rate over the life of the option and the put like a **floor**; B3 itself describes these options as instruments for protecting against interest-rate moves.

> **Remark 1** (*units*). Every monetary quantity returned by the engine (premium, vega) is in index points, which equal reais because the point value is R$ 1. The constant `IdiOption.PointValue` multiplies results in the engine so that the assumption is explicit and would change in one place if the contract multiplier changed.

3. From the DI curve to the forward of the IDI

3.1 What the curve gives us

The bootstrapped DI curve stores discount factors *P(0,T)*: the price today of R$ 1 paid at *T*. With the Bus/252 convention used throughout the library, a pillar quoted at annual zero rate z and DU business days away satisfies

$$P(0,T) = (1+z)^{-DU/252}, \qquad T = \frac{DU}{252} \tag{3}$$

Between pillars the curve is flat-forward (log-linear in the discount factor), which is the convention B3 uses for DI interpolation. In code: `market.Curve(CurveNames.Di).DiscountFactor(maturity)`.

3.2 The forward of the IDI by no-arbitrage

A **forward contract** on the IDI delivers IDI_T at time *T* for a price *F* agreed today. The IDI pays no income (it only accrues), so replication is a cash-and-carry:

- **Strategy A**: pay $F \times$ P(0,T) today into a zero-coupon bond that returns F at T, and enter the forward. At T you pay F and receive IDI_T.
- **Strategy B**: buy the IDI today for $S = \text{IDI}_0$. At T you own IDI_T.
- Both give IDI_T at T, so their cost today must match: $F \cdot P(0,T) = S$.

$$F = \frac{\text{IDI}_0}{P(0,T)} = \text{IDI}_0\,(1+z)^{DU/252} \tag{4}$$

This is the same identity the repository already tests for the CDI leg of the swap: with the same curve, the projected accrual factor over [0, T] equals 1/P(0,T) (see `ProjectedDailyRateIndex` tests). The engine therefore uses **one curve for two jobs**: to project the forward and to discount the option payoff. That consistency is not cosmetic; it is why the DV01 reported in Section 5.5 has a clean decomposition.

3.3 Worked numbers (from the actual code)

Valuation date 2026-09-25, DI1F28 maturity (2028-01-03), synthetic IDI level S = 1,000,000 points (only the ratio to the strike matters):

Table 3. Worked example, market inputs: valuation date 2026-09-25, maturity 2028-01-03, synthetic IDI level of 1,000,000 points.

Quantity	Value	How
Business days DU	317	CountBusinessDaysBetween(valuation, maturity)
Time T	1.257937 y	DU / 252
Zero rate z	13.9000 %	DI curve at the maturity pillar
Discount factor P(0,T)	0.84897860	$(1+z)^{-T}$
Forward F	1,177,885.99	S / P(0,T)

4. The Black-76 model

4.1 Assumptions

- **The forward is lognormal.** Under the pricing measure its log-return over [0,T] is normal; it can never be negative.
- **The forward is a martingale** in the measure whose numéraire is the zero-coupon bond P(·,T): no drift, because a forward price is already the expected future value.
- **Discounting is deterministic** for the option: one number, P(0,T), taken from the curve.
- **One volatility σ per option** (constant until T). Smile and term structure enter by choosing σ for each (T, K) from a surface – Section 8.

4.2 The dynamics

With forward F_t for delivery at T, Black's model (Black, 1976) is

$$dF_t = \sigma F_t\,dW_t \;\Rightarrow\; F_T = F_0 \exp\!\left(-\tfrac{1}{2}\sigma^2 T + \sigma\sqrt{T}\,Z\right), \quad Z \sim N(0,1) \tag{5}$$

The $-\tfrac{1}{2}\sigma^2 T$ term is not a modelling choice: it is exactly what makes $E[F_T] = F_0$ (martingale). The Monte Carlo test in Section 10 simulates this formula directly.

4.3 The price

The value today of a payoff paid at T is its expectation times the discount factor:

$$V_{\text{call}} = P(0,T)\,\mathrm{E}[(F_T - K)^+] = P(0,T)(F\,N(d_1) - K\,N(d_2)) \tag{6}$$

$$d_1 = \frac{\ln(F/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T} \tag{7}$$

where N is the standard normal CDF. Appendix A derives it in four lines. The two terms have a direct meaning:

- **$N(d_2)$** is the probability, under the T-forward measure, that the option finishes in the money: $P(F_T > K)$.
- **$F \cdot N(d_1)$** is the expected value of the forward *on the event* that it finishes above the strike, $E[F_T \cdot 1_{F_T>K}]$. $N(d_1)$ is the same probability under a different measure (the one where the forward itself is the numéraire).

4.4 Puts, and the sign trick

$$V_{\text{put}} = P(0,T)(KN(-d_2) - FN(-d_1)) \tag{8}$$

Call and put are one formula with a sign $w = +1$ (call) or -1 (put): `V = P · w · (F·N(w·d1) − K·N(w·d2))`. This is how `Black76.Price` is written; there is no separate branch for puts. Subtracting the two prices gives **put–call parity**, which holds with no model assumption at all:

$$V_{\text{call}} - V_{\text{put}} = P(0,T)\,(F - K) \;=\; \text{IDI}_0 - KP(0,T) \tag{9}$$

The right-hand form uses $F = S/P$ and is what the engine test checks, in points, for three strikes.

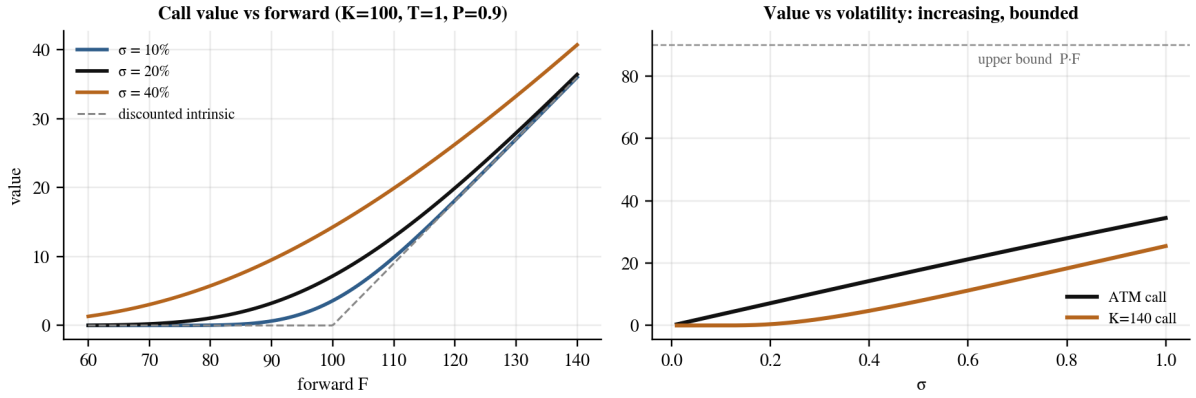


Figure 3. Left: call value as the forward moves, for three volatilities (the dashed line is intrinsic value; the gap above it is time value). Right: value increases with volatility and is capped by P·F (the call can never be worth more than the discounted forward).

4.5 Boundary behaviour (and what the code does there)

Table 4. Behaviour of the Black-76 price at the boundaries of its domain.

Case	Mathematical limit	In Black76.Price
$\sigma \to 0$ or $T \to 0$	value → discounted intrinsic, $P \cdot \max(w(F-K), 0)$	explicit early return; avoids division by $\sigma\sqrt{T} = 0$
$F \gg K$ (deep ITM call)	$N(d_1), N(d_2) \to 1$, value → $P(F-K)$	falls out of the formula
$F \ll K$ (deep OTM call)	value → 0	falls out; CDF tail computed accurately (Section 7)
$\sigma \to \infty$	call → $P \cdot F$, put → $P \cdot K$	the upper bounds used by the implied-vol solver

4.6 Black-76 versus Black–Scholes

Black–Scholes (Black and Scholes, 1973) takes a spot, an interest rate and a dividend yield and derives the forward internally as $S \cdot e^{(r-q)T}$. Black-76 takes the **forward and the discount factor as given**. For a Brazilian curve this is exactly what is wanted: the forward comes from the DI curve (Section 3), the discount factor comes from the same curve, and no flat 'r' or carry assumption has to be invented. The two models give identical prices when $F = S \cdot e^{(r-q)T}$ and $P = e^{-rT}$.

Remark 2 (*limitations of the model*). (1) Real option markets show a *smile*, so a single σ is wrong across strikes; we handle that by choosing σ from a surface, not by changing the model. (2) The DI curve is itself random, and the IDI forward and the discount factor move together; Black-76 treats P(0,T) as a constant. A model with stochastic rates (Hull–White, LMM) captures that; Black-76 is the market-standard first approximation. (3) It was not verified that B3's published reference premiums for IDI options are produced with Black-76; B3's pricing manual (B3, 2026b) documents the volatility calculation for options on DI1 futures, and should be consulted before claiming a reconciliation.

5. Greeks and rate risk

Greeks are partial derivatives of the price. They answer 'how much do I make or lose if one input moves?' and they are what a desk actually trades and hedges against. The module returns three, plus rate risk through the existing risk engine.

5.1 Greeks with respect to the forward

Table 5. Closed-form Greeks with respect to the forward (φ is the standard normal density).

Greek	Definition	Closed form (call)	Closed form (put)
Delta	$\partial V/\partial F$	$P \cdot N(d_1)$	$-P \cdot N(-d_1)$
Gamma	$\partial^2 V/\partial F^2$	$P \cdot \varphi(d_1) / (F\,\sigma\,\sqrt{T})$	same as call
Vega	$\partial V/\partial \sigma$	$P \cdot F \cdot \varphi(d_1) \cdot \sqrt{T}$	same as call

φ is the normal density. The derivations rest on one identity: $\mathbf{F \cdot \varphi(d_1) = K \cdot \varphi(d_2)}$. Because $d_2 = d_1 - \sigma\sqrt{T}$, $\varphi(d_2) = \varphi(d_1) \cdot \exp(d_1 \sigma\sqrt{T} - \tfrac{1}{2}\sigma^2 T) = \varphi(d_1) \cdot F/K$. When the product rule is applied to $V = P(F\,N(d_1) - K\,N(d_2))$, all the terms containing φ cancel and only $P \cdot N(d_1)$ is left for delta; for vega the identity collapses the result to $P \cdot F \cdot \varphi(d_1) \cdot \sqrt{T}$ (Appendix A).

5.2 Greeks in terms of the IDI level

A trader holds the option against the *index*, not against the abstract forward. Since F = S/P, the chain rule gives

$$\frac{\partial V}{\partial S} = \frac{1}{P}\frac{\partial V}{\partial F} = N(d_1), \qquad \frac{\partial^2 V}{\partial S^2} = \frac{1}{P^2}\frac{\partial^2 V}{\partial F^2} = \frac{\varphi(d_1)}{S\sigma\sqrt{T}} \tag{10}$$

This is the conversion in `IdiOptionPricingEngine.Greeks`: `g.Delta / Discount` and `g.Gamma / Discount²`. Vega needs no conversion. A useful consequence: the *IDI delta of a call is just $N(d_1)$*, a number in [0,1] – the engine test checks both that range and that delta matches a ±100-point bump of the quote.

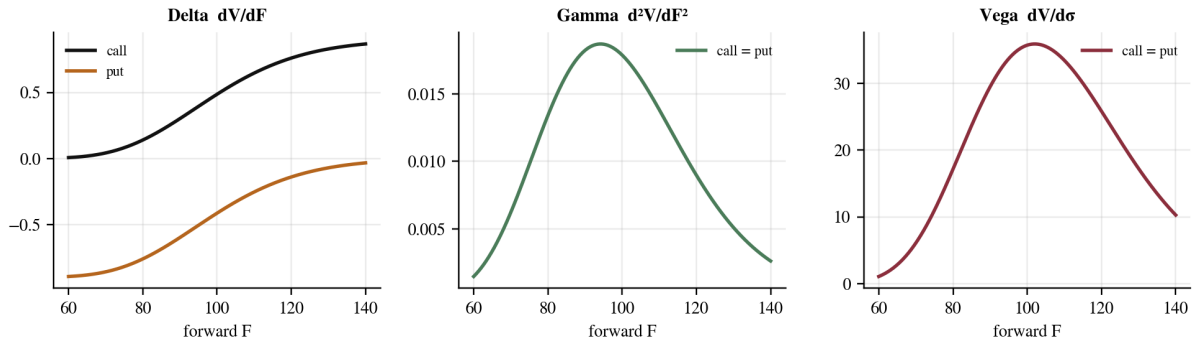


Figure 4. Greeks against the forward (K=100, T=1, σ=20%). Delta moves from 0 to 1 as the call goes in the money; gamma and vega peak near the strike and are identical for calls and puts.

5.3 Relationships worth knowing (and tested)

- $\mathbf{Delta_{call} - Delta_{put} = P}$ in forward terms, or 1 in index terms (derivative of put–call parity).
- **Gamma and vega are the same for call and put**, because parity differs only by a linear function of F, which has zero second derivative and zero σ-dependence.

- **Gamma and vega are largest at the money** and shrink for deep in/out of the money options – this is why vega → 0 breaks naive Newton in Section 6.

5.4 Sticky-strike convention

When the engine bumps the index to measure delta, the volatility is held fixed at the value read from the surface for the *current* log-moneyness. This is the **sticky-strike** convention: each strike keeps its own vol when spot moves. The alternative, sticky-moneyness, would re-read the surface after the bump and give a different delta. There is no 'right' answer; desks choose one and state it, and the module documents its choice in `IdiOptionGreeks`.

5.5 Interest-rate risk (DV01) through the existing engine

Because `IdiOptionPricingEngine` implements `ICurvePricingEngine<IdiOption>` and reads the curve from the market on every call, the repository's `CurveRiskEngine<IdiOption>` works unchanged: it bumps the DI curve by ±1bp, reprices, and reports (PV(−1bp) − PV(+1bp))/2. Both the forward and the discount factor move together under the bump.

At fixed IDI level and volatility, the dependence on rates is only through P = P(0,T):

$$V_{\text{call}} = SN(d_1) - KPN(d_2) \;\Rightarrow\; \frac{\partial V_{\text{call}}}{\partial P} = -KN(d_2) < 0 \tag{11}$$

Higher rates lower P, which *raises* the call (the strike is paid later, in cheaper money) and lowers the put. Under the library's DV01 convention (positive when value falls as rates rise) the call has **negative** DV01 and the put **positive**. Check with the worked example below: one basis point moves P by about $-P \cdot T/(1+z) \cdot 10^{-4} = -9.38 \cdot 10^{-5}$, so the put's DV01 $\approx K \cdot N(-d_2) \cdot 9.38 \cdot 10^{-5} = 1{,}300{,}000 \times 0.7223 \times 9.38 \cdot 10^{-5} \approx 88.1$, and the call's $\approx -1{,}300{,}000 \times 0.2777 \times 9.38 \cdot 10^{-5} \approx -33.9$. Both match the code.

5.6 Worked example (K = 1,300,000, σ = 18%, same market as Section 3.3)

Table 6. Worked example: K = 1,300,000, σ = 18%, same market as Table 3.

Output	Call	Put	Notes
d_1, d_2	−0.387670, −0.589554	same	F=1,177,885.99; $\sigma\sqrt{T} = 0.20188$
$N(d_1)$, $N(d_2)$	0.349130, 0.277745	—	P(finish ITM) ≈ 27.8% for the call
Premium (points = R$)	42,590.86	146,263.04	call − put = −103,672.18 = S − K·P
Delta (index)	0.349130	−0.650870	difference = 1, as parity requires
Gamma (per point)	$1.833 \cdot 10^{-6}$	same	
Vega (per 1.00 of σ)	415,054	same	≈ R$ 4,151 per vol point (1%)
DV01 (per bp)	−33.85	+88.04	signs explained in 5.5

Not implemented (deliberately, to keep scope clear): theta (needs a day-count and decay convention), rho in the Black sense (superseded by DV01 here), and cross-Greeks such as vanna and volga, which matter once a smile is calibrated.

6. Implied volatility

6.1 Why it matters

Options are quoted and compared in **premium**, but premiums are not comparable across strikes and maturities. The *implied volatility* – the σ that makes Black-76 reproduce a premium – puts every option on one scale, so it is the language in which surfaces are quoted, calibrated and checked. The engine method `ImpliedVolatility(option, market, premium)` turns a premium in points into σ.

6.2 Existence and uniqueness

Vega is strictly positive, so the price is strictly increasing in σ and the inverse is unique – if it exists. It exists only when the premium lies inside the no-arbitrage band, which is the range of prices that $\sigma \in (0, \infty)$ can produce:

$$P\max(w(F-K),0) < V < PF \text{ (call)}, \qquad P\max(w(F-K),0) < V < PK \text{ (put)} \tag{12}$$

Outside the band no volatility exists (the premium is below intrinsic value or above the model's cap), and the code throws `ArgumentException` with the allowed range instead of returning a meaningless number. A quote outside the band is a data problem that deserves a loud failure.

6.3 The algorithm: Newton with a bisection safeguard

Newton's method on f(σ) = V(σ) − target uses the slope that we already have in closed form, vega: $\sigma_{n+1} = \sigma_n$ − f/vega. It converges quadratically near the root but can fail badly far from it: for deep out-of-the-money options vega is almost zero, so the step f/vega is huge and can throw σ negative or to a absurd magnitude. The code therefore keeps a bracket [lo, hi] known to contain the root (initially $[10^{-8}, 10]$) and, at each iteration:

- evaluates the price; if it is above the target, hi = σ; otherwise lo = σ (valid because price is increasing in σ);
- tries the Newton step; **accepts it only if it falls strictly inside (lo, hi)**;
- otherwise takes the midpoint (bisection), which always halves the bracket;
- stops when $|\text{error}| \leq 10^{-13} \times$ price (relative, because premiums here are in the millions) or the bracket is narrower than 10^{-15}.

This is the standard safeguarded Newton (hybrid Newton–bisection) scheme (Press et al., 2007): the speed of Newton when it behaves and the guaranteed convergence of bisection when it does not. Jäckel (2015) gives a non-iterative alternative whose accuracy approaches machine precision; it is not used here.

Tables 7 and 8 trace the algorithm (a Python replica of the C# loop; F=100, T=1.5, P=0.92, starting guess σ=0.20). Case A is an ordinary near-the-money option; Case B is deep out of the money, where Newton's first steps would be unsafe:

Table 7. Solver trace, Case A: call, K = 105, true σ = 30%.

Iter	σ tried	price − target	next step
0	`0.2000000000`	`-4.492e+00`	Newton
1	`0.3002240215`	`+1.006e-02`	Newton
2	`0.2999999986`	`-6.068e-08`	Newton
3	`0.3000000000`	`+0.000e+00`	**converged**

Table 8. Solver trace, Case B: call, K = 300, true σ = 90%. The first steps are bisections because vega at the start is nearly zero.

Iter	σ tried	price − target	next step
0	`0.2000000000`	`-1.338e+01`	bisection
1	`5.1000000000`	`+7.834e+01`	bisection
2	`2.6500000000`	`+6.258e+01`	bisection
3	`1.4250000000`	`+2.309e+01`	Newton
4	`0.8958987917`	`-1.667e-01`	Newton
5	`0.9000065449`	`+2.664e-04`	Newton
6	`0.9000000000`	`+6.680e-10`	Newton
7	`0.9000000000`	`+7.105e-15`	**converged**

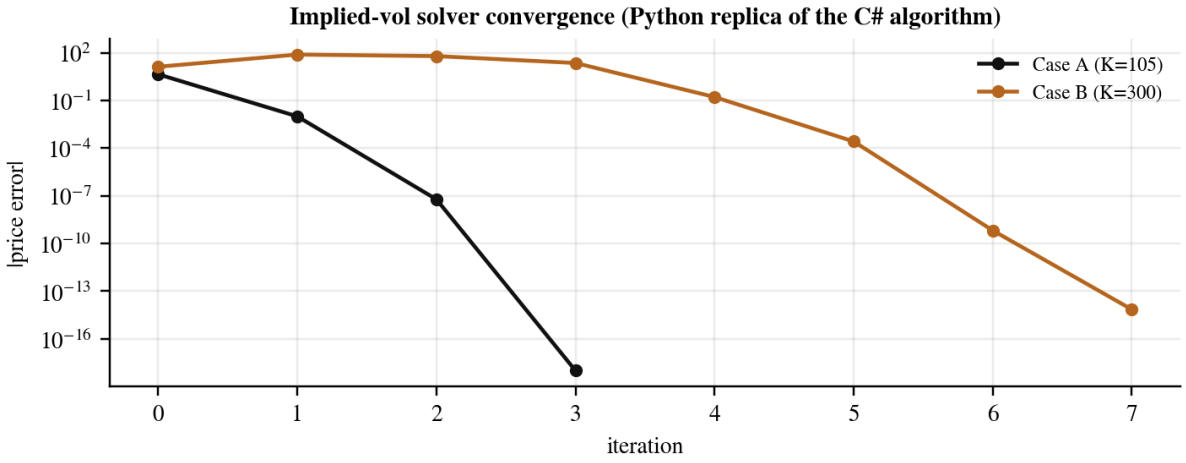


Figure 5. Price error per iteration on a log scale. Case B starts with bisection steps that shrink the bracket; Newton then takes over and converges in a few steps.

Round-trip test. The test suite prices an option at a known σ and checks that the solver recovers it to 10^{-9} for five (type, strike, σ) combinations from σ=5% to 75% and strikes from deep ITM to far OTM; the engine test repeats it through the full engine for a put.

7. The normal distribution function

Everything above needs $N(x) = P(Z \le x)$ for a standard normal Z. It has no closed form, but it is related to the complementary error function:

$$N(x) = \tfrac{1}{2}\,\mathrm{erfc}\left(-\frac{x}{\sqrt{2}}\right) \tag{13}$$

.NET's base library does not ship erfc or a normal CDF, and the library has no third-party dependencies by design, so the module implements N itself.

7.1 Why accuracy matters here

The classical Abramowitz–Stegun polynomial approximation (Abramowitz and Stegun, 1964) has an absolute error near 10^{-7}. That is fine for a rough price, but this module *differentiates* prices to test Greeks (central finite differences divide price differences by small steps) and inverts the price to get implied vol to 10^{-9}. A 10^{-7} error in N would be amplified and make those tests meaningless. The implementation is therefore Hart's double-precision rational approximation (Hart et al., 1968) as tuned by West (2005): a degree-6/degree-7 rational function for $|x| < 7.07$ and a continued fraction beyond.

7.2 Computing from the small side

The code evaluates the *tail* – the small probability beyond |x| – and then returns tail for $x < 0$ or $1 - \text{tail}$ for $x > 0$. Computing $N(-8)$ as $1 - N(8)$ would subtract two numbers that agree in 15 digits and lose all precision; computing the tail directly keeps full *relative* accuracy even when N(x) is 10^{-16}. This matters for deep out-of-the-money options, whose prices are tails. Beyond $|x| = 37$ the result is exactly 0 or 1 in double precision.

7.3 How it is tested

`NormalDistributionTests` checks N(x) at 0, ±1, 1.96, −3 and −8 against reference values (to 10^{-9} relative, and 10^{-14} absolute), symmetry $N(x) + N(-x) = 1$ at three points, and that the density is the derivative of N (finite difference). A mistyped coefficient will show up in these tests, and the tail cases (−3, −8) are the most sensitive, so type them carefully.

> **Remark 3** (*implementation*). The coefficients are taken from the literature and cannot be derived by hand; the tests above exist to catch transcription errors.

8. The volatility surface

8.1 Why a surface

Black-76 has one σ, but the market does not agree: options at different strikes (the *smile* or *skew*) and different maturities (the *term structure*) trade at different implied vols. The standard fix is not to abandon Black-76 but to feed it the **right σ for each option**, read from a surface σ(T, K) (Gatheral, 2006). The module defines the contract `IVolSurface` and two implementations.

8.2 Coordinates: time and log-moneyness

The surface is indexed by time to expiry T (in years, Bus/252) and by **log-moneyness** k = ln(K/F), not by the raw strike. Reason: when the forward moves, a given strike is more or less in the money; the market quotes 'ATM vol', '25-delta vol' and so on, which are statements about moneyness. Using k keeps the surface meaningful when F changes, and at k = 0 it returns the at-the-money vol.

8.3 Interpolating total variance, not volatility

Between two quoted expiries the surface interpolates **total variance** $w = \sigma^2 T$ linearly in T, then converts back to $\sigma = \sqrt{(w/T)}$. Why variance and not vol? Variance adds up over time (for independent increments it is the sum over sub-periods), so linear-in-w is the interpolation consistent with how risk accumulates, and it guarantees w stays non-decreasing if the node values are.

$$w(T,k) = w_i(k) + \frac{T - T_i}{T_{i+1} - T_i}(w_{i+1}(k) - w_i(k)), \qquad \sigma(T,k) = \sqrt{w(T,k)/T} \tag{14}$$

Within an expiry the quoted vols are interpolated **linearly in k** and held flat beyond the first and last quoted strikes. Before the first expiry or after the last, σ is held flat (not w), a conservative choice that avoids inventing term structure.

8.4 No-arbitrage conditions

A surface that violates no-arbitrage conditions can produce negative prices for some portfolio of options. Two classical conditions:

- **Calendar spread**: at a fixed log-moneyness, total variance must not decrease with maturity. Intuitively, a longer option on the same forward cannot be worth less than a shorter one, and the Black price (per unit of forward) is increasing in w. **Implemented**: the `GridVolSurface` constructor checks $w_{i+1}(k) \geq w_i(k)$ at every node and throws otherwise.
- **Butterfly**: the option price must be convex in strike (the implied risk-neutral density must be non-negative). **Not implemented.** Linear interpolation in k can violate it near kinks; the standard remedies are arbitrage-free parametrisations such as SVI (Gatheral and Jacquier, 2014) or SABR (Hagan et al., 2002) (Section 11).

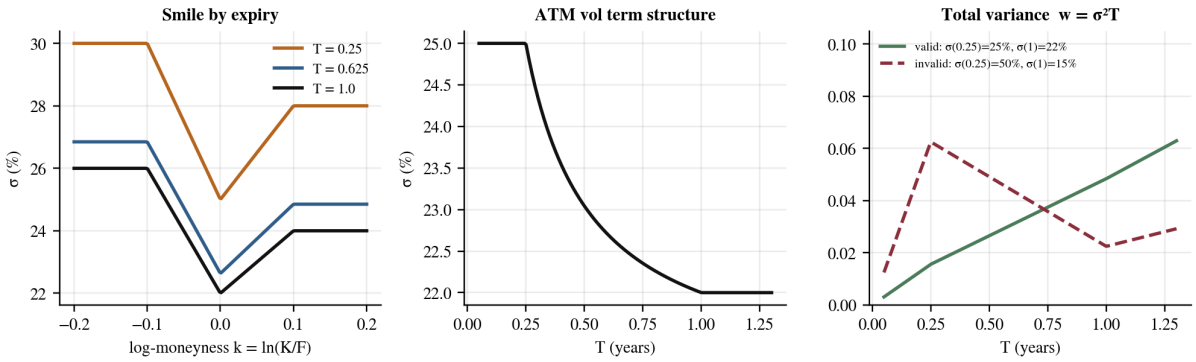


Figure 6. Left: smile slices; interpolation in w gives the intermediate T = 0.625 slice. Middle: ATM vol term structure. Right: total variance. The red dashed curve is a surface the constructor rejects, because w falls from 0.0625 at T=0.25 to 0.0225 at T=1.

8.5 Validation performed by the constructor

- expiries strictly increasing and positive; log-moneyness strictly increasing;

- one row per expiry and one column per strike; all vols positive;
- calendar-spread condition at every strike node.

`FlatVolSurface` is the trivial case, a constant σ. It is what most unit tests use, because it removes the surface as a source of differences when testing the engine, and the engine test with a skewed grid shows that a skew raises the price of an out-of-the-money put as expected.

9. The pricing engine

`IdiOptionPricingEngine` is the glue: it turns a `MarketContext` and an `IdiOption` into the five numbers Black76 needs. Its private `Inputs` method does the work:

```
double spot     = (double)market.Quote(QuoteNames.Idi);                // IDI level, in points
double discount = market.Curve(CurveName).DiscountFactor(option.MaturityDate);   // P(0,T)
double time     = market.DayCount.YearFraction(
                      market.ValuationDate, option.MaturityDate, market.Calendar);  // DU / 252
double forward  = spot / discount;                                     // F = S / P(0,T)
double vol      = time > 0 ? _surface.Volatility(time, Math.Log(option.Strike / forward)) : 0.0;
```

9.1 Responsibilities

- **Price** = `Greeks(...).Price`: one code path for price and Greeks, so they cannot disagree.
- **Greeks** calls Black76.Greeks and converts forward-Greeks to index-Greeks (Section 5.2), multiplying by `PointValue` where relevant.
- **ImpliedVolatility** recovers σ from a premium; it ignores the surface's σ (it is computed but unused) because the answer depends only on forward, strike, time, discount and type.

9.2 The ICurvePricingEngine contract

The interface `ICurvePricingEngine<T>` adds one member, `CurveName`, to `IPricingEngine<T>`. It tells risk engines which curve to bump. The matching rule, stated in a comment on `QuoteRiskEngine`, is: **the engine must read the curve from the market on every `Price` call**. If an engine cached a curve or a derived object at construction, a bumped market would never reach it and the DV01 would silently be zero. Here the curve is read inside `Inputs` each call, so a bump moves both F and P.

9.3 Edge cases

Table 9. Engine behaviour in edge cases.

Situation	Behaviour	Why
Valuation after maturity	returns zeros	the option no longer exists; also avoids asking the curve for a date before its reference
Valuation on the maturity date	$T = 0$, $\sigma = 0 \rightarrow$ intrinsic value	Black76 handles $T \leq 0$ explicitly; DF(maturity) = 1
IDI quote missing	KeyNotFoundException from MarketContext	the context names the missing quote and lists the available ones
Implied vol of an impossible premium	ArgumentException with the allowed range	see Section 6.2

9.4 A design trade-off: where does the surface live?

The vol surface is passed to the engine constructor, not stored in the `MarketContext`. That keeps the Core project untouched (the context knows curves, indices and decimal quotes; `IVolSurface` lives in the Derivatives project, and Core cannot reference it). The cost is that a 'vol bump' scenario creates a new engine rather than a new context, unlike curve scenarios. A production design usually moves the interface to Core and adds `WithVolSurface` to the context, so that all market data – including vol – is one immutable snapshot. It is a good follow-up exercise.

10. Testing strategy

Pricing code fails silently: a wrong formula still returns a plausible number. The tests therefore use **several independent kinds of evidence**, so that a bug has to fool all of them to survive. There are 46 tests in `FixedIncome.Derivatives.Tests`.

Table 10. Verification techniques used in the test suite and the defects each is designed to detect.

Technique	Example in this module	Bug it would catch
Reference values	$N(1.96) = 0.97500210\ldots$; ATM Black price 7.9655674554	typo in a CDF coefficient; wrong d_1
Mathematical identities	put–call parity (model and engine, three strikes); $\text{delta}_c - \text{delta}_p = 1$; equal gamma and vega	wrong sign on the put branch; inconsistent d_1/d_2
Finite differences	delta, gamma, vega vs central differences of Price; engine delta vs ±100-point bump of the quote	a Greek formula that does not match the price function
Round trip	$\text{Price}(\sigma) \to \text{ImpliedVolatility} \to \sigma$ for 5 cases; engine round trip	a solver that stops early or diverges
Independent computation	Monte Carlo (below)	an error shared by price and Greeks, such as the wrong drift
Bounds and monotonicity	price increasing in σ, within [P·intrinsic, P·F]; solver rejects out-of-band premiums	violations of no-arbitrage
Structural / constructor checks	surface rejects calendar arbitrage and malformed grids; strike must be positive	garbage in
Integration	engine price equals Black76 on the curve-implied forward; CurveRiskEngine DV01 signs	wrong forward, wrong DF, an engine that caches the curve

10.1 The Monte Carlo test, explained

The closed form is checked against a simulation that never uses $N(\cdot)$ (Glasserman, 2004). Using the dynamics of Section 4.2, one draws $Z \sim N(0,1)$ via Box–Muller, forms $F_T = F \cdot \exp(-\tfrac{1}{2}\sigma^2 T + \sigma\sqrt{T} \cdot Z)$, averages the discounted payoff $P \cdot \max(F_T - K, 0)$, and compares with Black76.Price:

$$\hat{V} = \frac{1}{n}\sum_{i=1}^{n} P \max(F_T^{(i)} - K, 0), \qquad \text{SE} = \frac{s}{\sqrt{n}} \tag{15}$$

The assertion allows 4 standard errors: if the closed form were off by more than the sampling noise, the test fails; a correct formula fails only about 1 time in 15,000. The generator uses a fixed seed, so the test is deterministic; **antithetic variates** (using both Z and −Z) reduce variance, and the standard error is computed treating all draws as independent, which is slightly conservative.

> **Remark 4** (*value of independent checks*). Parity and finite-difference tests check the formula against *itself*. If d_1 had the wrong sign of the $\sigma^2 T/2$ term everywhere, parity would still hold, and finite differences would still match. Only a simulation that starts from the model's assumptions – the lognormal martingale – can expose that kind of error.

11. Limitations and next steps

11.1 What this module does not do yet

- **No source for the IDI level.** The tests use a synthetic level. Pricing real contracts needs B3's daily IDI value (or the CDI series accumulated back to the index base).
- **No B3 reconciliation.** The repository's other products are validated against ANBIMA/B3 official numbers. For IDI options I could not confirm a downloadable file of reference premiums or vols; B3's option pricing manual (B3, 2026b) and market-data files should be consulted before choosing the validation target.
- **No ticker or calendar logic** for contract months (compare `DiFuture.FromTicker`).
- **No smile model.** The grid interpolates linearly in k and checks calendar but not butterfly arbitrage.

- **Options on DI1 futures** (strike set as a rate, exercise into a futures position) are a different contract, not covered here.
- **No theta, rho or second-order Greeks.**

11.2 Future work

Table 11. Proposed extensions.

#	Exercise	What it teaches
1	Add the IDI quote and an option to the Sandbox book; show price, Greeks and DV01 in the report	wiring a new product through market data, portfolio and reporting
2	Move `IVolSurface` to Core and add `WithVolSurface` to MarketContext; add a 'vol +1 point' scenario	scenario design; the cost of where state lives
3	Fit an SVI or SABR slice to a handful of quoted vols; add a butterfly-arbitrage check	calibration, parametric smiles, arbitrage-free surfaces
4	Parse B3 option reference data and reconcile premiums / implied vols	model validation against market data
5	Hull–White one-factor model calibrated to the DI curve; price a European option on a zero-coupon bond analytically and with a tree	stochastic rates; comparing two numerical methods

12. Conclusion

The module prices the B3 option on the IDI index with Black (1976) applied to the forward implied by the DI curve, and returns delta, gamma, vega, DV01 and implied volatility. The same curve projects the forward and discounts the payoff, so the library's existing risk engine reports rate sensitivity without modification. A volatility surface interpolated in total variance, with a calendar-arbitrage check, supplies the volatility for each strike and maturity.

Correctness rests on independent evidence rather than on any single check: closed-form reference values, put–call parity, finite-difference agreement for every Greek, round-trip recovery of implied volatility, and agreement with a Monte Carlo simulation of the underlying lognormal martingale. What remains open is validation against market data: a source for the IDI level and a reconciliation of premiums and implied volatilities with B3 (Section 11).

References

Abramowitz, M. and Stegun, I. A. (eds.) (1964). *Handbook of Mathematical Functions*. National Bureau of Standards, Washington, DC.

B3 (2026a). *Opções sobre IDI* (product page). https://www.b3.com.br/pt_br/produtos-e-servicos/negociacao/juros/opcoes-sobre-idi.htm

B3 (2026b). *Manual de Apreçamento – Opções*. https://www.b3.com.br/data/files/ED/75/F7/EC/78003710F24CFF27AC094EA8/Manual_de_Aprecamento_-_Opcoes.pdf

Black, F. (1976). The pricing of commodity contracts. *Journal of Financial Economics*, 3(1–2), 167–179.

Black, F. and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.

Brigo, D. and Mercurio, F. (2006). *Interest Rate Models – Theory and Practice* (2nd ed.). Springer.

Gatheral, J. (2006). *The Volatility Surface: A Practitioner's Guide*. Wiley.

Gatheral, J. and Jacquier, A. (2014). Arbitrage-free SVI volatility surfaces. *Quantitative Finance*, 14(1), 59–71.

Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer.

Hagan, P. S., Kumar, D., Lesniewski, A. S. and Woodward, D. E. (2002). Managing smile risk. *Wilmott Magazine*, September, 84–108.

Hart, J. F., Cheney, E. W., Lawson, C. L., Maehly, H. J., Mesztenyi, C. K., Rice, J. R., Thacher, H. C. and Witzgall, C. (1968). *Computer Approximations*. Wiley.

Jäckel, P. (2015). Let's be rational. *Wilmott Magazine*, 2015(75), 40–53.

Press, W. H., Teukolsky, S. A., Vetterling, W. T. and Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press.

West, G. (2005). Better approximations to cumulative normal functions. *Wilmott Magazine*, May, 70–76.

Appendix A. Derivations

A.1 The Black-76 call price

Let $m = \ln(F/K)$, $s = \sigma\sqrt{T}$ and $Z \sim N(0,1)$, so $F_T = F\cdot\exp(-s^2/2 + sZ)$. Then

$$F_T > K \iff Z > -d_2, \qquad d_2 = \frac{m - \frac{1}{2}s^2}{s} \tag{16}$$

The strike term is a probability: $E[K\cdot 1_{F_T>K}] = K\cdot P(Z > -d_2) = K\cdot N(d_2)$. For the forward term complete the square in the exponent:

$$\mathrm{E}[F_T \mathbf{1}_{F_T>K}] = F\int_{-d_2}^{\infty} e^{sz-\frac{1}{2}s^2}\varphi(z)\,dz = F\int_{-d_2}^{\infty}\varphi(z-s)\,dz = F\,N(d_2+s) = F\,N(d_1) \tag{17}$$

Subtracting and multiplying by the discount factor gives $V = P\cdot(F\cdot N(d_1) - K\cdot N(d_2))$. The put follows from parity, or by the same argument on the event $Z < -d_2$.

A.2 Delta, gamma, vega

From $d_1 = \ln(F/K)/(\sigma\sqrt{T}) + \sigma\sqrt{T}/2$ we get $\partial d_1/\partial F = \partial d_2/\partial F = 1/(F\sigma\sqrt{T})$. Using $F\cdot\varphi(d_1) = K\cdot\varphi(d_2)$:

$$\frac{\partial}{\partial F}[FN(d_1) - KN(d_2)] = N(d_1) + \frac{F\varphi(d_1) - K\varphi(d_2)}{F\sigma\sqrt{T}} = N(d_1) \tag{18}$$

$$\frac{\partial^2 V}{\partial F^2} = P\,\varphi(d_1)\frac{1}{F\sigma\sqrt{T}}, \qquad \frac{\partial V}{\partial \sigma} = P\,F\varphi(d_1)\left(\frac{\partial d_1}{\partial \sigma} - \frac{\partial d_2}{\partial \sigma}\right) = P\,F\varphi(d_1)\sqrt{T} \tag{19}$$

A.3 Why the calendar-spread condition is w non-decreasing

Per unit of forward and without discounting, a call at log-moneyness k is $c(k,w) = N(d_1) - e^k N(d_2)$ with $d_{1,2} = (-k \pm w/2)/\sqrt{w}$, which depends on (σ,T) only through $w = \sigma^2 T$ and is increasing in w (its derivative is $\varphi(d_1)/(2\sqrt{w}) > 0$). Therefore a longer-dated option at the same moneyness is worth at least as much as a shorter one only if w does not decrease; otherwise a calendar spread would have negative value.

Appendix B. Formula summary

Table 12. Formula summary.

Item	Formula
Forward of IDI	$F = IDI_0 / P(0,T)$
Time	$T = DU / 252$ (Bus/252)
d_1, d_2	$d_1 = [\ln(F/K) + \tfrac{1}{2}\sigma^2 T] / (\sigma\sqrt{T})$; $d_2 = d_1 - \sigma\sqrt{T}$
Call / put	$V = P\cdot w\cdot[F\cdot N(w\,d_1) - K\cdot N(w\,d_2)]$, $w = +1 / -1$
Parity	$C - P = P(0,T)(F - K) = IDI_0 - K\cdot P(0,T)$
Delta (forward / index)	$P\cdot w\cdot N(w\,d_1)$ / $w\cdot N(w\,d_1)$
Gamma (forward / index)	$P\cdot\varphi(d_1)/(F\,\sigma\sqrt{T})$ / $\varphi(d_1)/(S\,\sigma\sqrt{T})$
Vega	$P\cdot F\cdot\varphi(d_1)\cdot\sqrt{T}$
$\partial V/\partial P$ at fixed S	call: $-K\cdot N(d_2)$; put: $+K\cdot N(-d_2)$
Total variance	$w = \sigma^2 T$; interpolate linearly in T at fixed k
Log-moneyness	$k = \ln(K/F)$

Appendix C. Glossary of Brazilian market terms

Table 13. Glossary of Brazilian market terms.

Term	Meaning
DI / CDI	Overnight interbank deposit rate, the Brazilian risk-free benchmark (annualised, Bus/252).
IDI	Índice de Taxa Média de Depósitos Interfinanceiros de Um Dia: DI accumulated daily, starting at 100,000 points.
Prêmio	Option premium, here quoted in IDI points.
Preço de exercício	Strike, in IDI points.
Vencimento	Maturity; for IDI options the first business day of the contract month.
PU (preço unitário)	Unit price of a contract; for DI1 futures $100{,}000 / (1+\text{rate})^{DU/252}$.
Dias úteis (DU)	Business days according to the B3/ANBIMA calendar.
Preço de ajuste	Daily B3 settlement price used for marking futures and for the DI curve.